CS2023 - Programación III

Ejercicios de patrones de diseño en C++20

Rubén Rivas Medina

Octubre 2026

Ejercicios: patrones de diseño en C++20

Implementar 3 programas pequeños que usen los patrones de la sesión: `Factory Method`, `Builder`, `Adapter`, `Decorator`, `Strategy` y `Observer`.

Priorizar C++20: `concept`, templates, lambdas, `std::function`, `std::ranges`, `std::optional`, `std::variant` y RAII. Usar herencia virtual sólo si se necesita polimorfismo en tiempo de ejecución.

Los esqueletos deben seguir el estilo de las diapositivas: contratos con `concept`, fábricas tipadas con `template<Concept T, class... Args>`, builders con punteros a miembros, decoradores `Decorator<T>` y estrategias como callables.

Organización del documento

Este documento reúne los enunciados actualizados de las carpetas 3, 4 y 5 de la unidad: Cafetería, Gastos personales y Sensores inteligentes. Aquí se numeran como ejercicios 1, 2 y 3.

Cada enunciado presenta el escenario, el funcionamiento, los diagramas de referencia con roles de los patrones, un ejemplo completo de uso, el esqueleto C++20, los contratos de implementación, las pruebas y los entregables.

Distribución obligatoria

Ejercicio	Creacional	Estructural	Comportamiento
1. Cafetería	`Builder`	`Adapter`	`Observer`
2. Gastos personales	`Factory Method`	`Decorator`	`Strategy`
3. Sensores inteligentes	`Factory Method`, `Builder`	`Adapter`, `Decorator`	`Strategy`, `Observer`

Entregables generales

- Código C++20 por ejercicio, separado en archivos claros.
- `main.cpp` con un flujo demostrativo.
- Como mínimo, 3 pruebas para Cafetería, 5 para Gastos personales y 6 para Sensores inteligentes, según sus entregables. Ejecutar también los casos adicionales necesarios para verificar los contratos.
- Sin `new`/`delete` directos; usar valores, RAII y smart pointers cuando corresponda.
- Breve respuesta: ¿qué patrón desacopla qué parte del diseño?

Ejercicio 1: Cafetería

Escenario de uso
Se desea construir un sistema simple para registrar pedidos en una cafetería. Cada pedido incluye cliente, bebida, tamaño y extras opcionales. Al confirmar el pedido, el sistema debe imprimir un ticket y avisar a otros componentes, como cocina, ventas o registro de eventos.

El diseño debe permitir cambiar la impresora, agregar nuevas pantallas o añadir efectos al confirmar un pedido sin modificar el flujo principal de `OrderService`.

Cómo funciona
Para el escenario inicial, Ana pide un latte grande con leche de avena y canela. El programa construye el pedido, imprime un ticket y comunica su confirmación.

- **Pedido (`DrinkOrder`):** reúne cliente, bebida, tamaño, extras y elección de leche de avena.
- **Constructor (`Builder<DrinkOrder>`):** permite completar esos campos con llamadas encadenadas a `set`.
- **Impresora antigua (`LegacyTicketPrinter`):** solo recibe texto mediante `plain_old_format`.
- **Adaptador (`TicketPrinterAdapter`):** convierte el pedido en texto y cumple el contrato `TicketPrinter`.
- **Servicio (`OrderService`):** coordina la impresión y la publicación del evento.
- **Bus (`EventBus`):** conserva suscripciones y ejecuta los callbacks al recibir un `OrderCreatedEvent`.
- **Cocina, ventas y log:** observadores que reaccionan a la confirmación.

Asociar el servicio con la impresora y el bus
En `OrderService service{printer, events}`, el constructor guarda referencias a los mismos objetos creados en `main`. Los observadores se conectan a ese `events` con `subscribe(handler)`. Cada handler recibe `const OrderCreatedEvent&` y retorna `void`; este ejercicio usa una sola forma de suscripción, sin filtros.

Al ejecutar `service.confirm(order)`, primero se imprime el ticket y después se publica un evento. El bus llama a cocina, ventas y log en orden de suscripción. El servicio no conoce esos callbacks ni llama a cada uno directamente.

Diagrama de referencia y roles

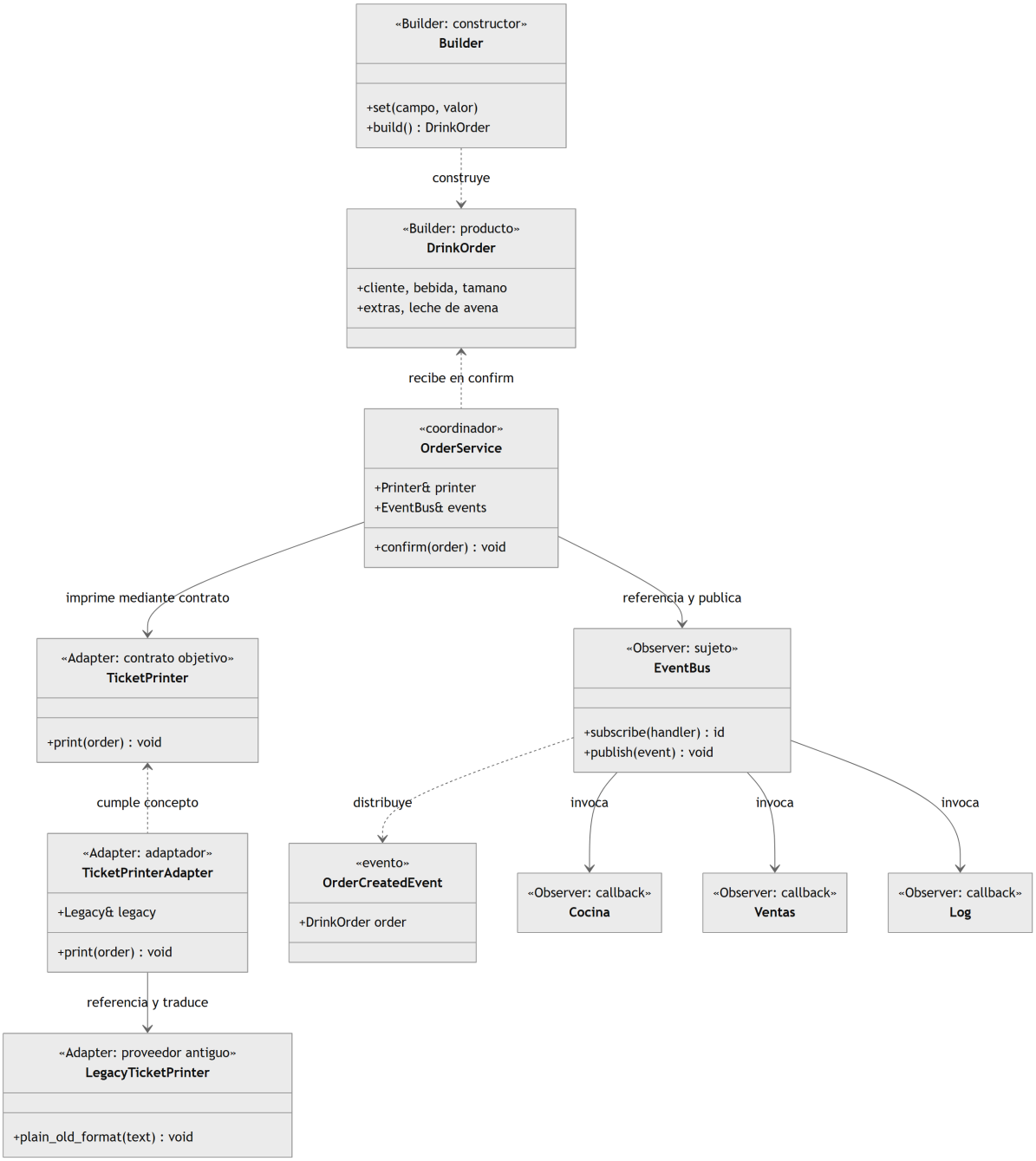


Figura 1: Builder prepara el pedido; OrderService usa un Adapter para imprimir y un bus Observer para notificar.

- **Builder:** `Builder<DrinkOrder>` es el constructor, `DrinkOrder` es el producto y `main` encadena los pasos.
- **Adapter:** `TicketPrinter` es el contrato objetivo; `LegacyTicketPrinter` es el componente con interfaz incompatible; `TicketPrinterAdapter` traduce entre ambos.
- **Observer:** `EventBus` es el sujeto que administra suscripciones; cocina, ventas y log son observadores; `OrderCreatedEvent` contiene la novedad. `OrderService` origina la publicación.

Los recuadros de conceptos y callbacks representan roles, no clases adicionales que deban implementarse. Las relaciones que indican cumplimiento de un concepto no exigen herencia.

Para qué sirve cada patrón
Builder separa la preparación del pedido de su confirmación. Adapter permite reutilizar la impresora antigua sin que el servicio dependa de ella. Observer permite agregar nuevos receptores sin modificar `OrderService`.

Patrones requeridos

`Builder`	Construye `DrinkOrder`: cliente, bebida, tamaño, leche vegetal y extras.
`Adapter`	Adapta `LegacyTicketPrinter` al contrato moderno `TicketPrinter`.
`Observer`	Publica `OrderCreatedEvent` hacia cocina, ventas y log.

Reglas del dominio
- Cliente y bebida deben ser no vacíos; cada extra agregado debe ser no vacío. El tamaño debe ser uno de los tres valores de `Size`.
- Los extras son opcionales y `oat_milk` vale `false` por defecto.
- Rechazar pedidos inválidos con `std::invalid_argument` al construirlos y al confirmarlos, antes de imprimir o notificar.
- Cada confirmación válida imprime una vez y publica un evento con una copia del pedido.
- Confirmar dos veces el mismo pedido representa dos operaciones; no se requiere detectar duplicados.
- Si la impresión lanza una excepción, propagarla sin publicar el evento. No se requiere deshacer una impresión parcial.

Funcionamiento completo antes de implementar
El ejemplo muestra el uso esperado una vez completadas las declaraciones del esqueleto.

Preparar el pedido

```
#include "orders.h"
#include <iostream>

int main() {
    auto order = Builder<DrinkOrder>{}
        .set(&DrinkOrder::customer_name, "Ana")
        .set(&DrinkOrder::drink, "Latte")
        .set(&DrinkOrder::size, Size::Large)
        .set(&DrinkOrder::oat_milk, true)
        .set(&DrinkOrder::extras,
             std::vector<std::string>{"cinnamon"})
        .build();
```

Conectar la impresora y los observadores
Continuación del mismo `main`:

```
    LegacyTicketPrinter legacy{std::cout};
    TicketPrinterAdapter<LegacyTicketPrinter, DrinkOrder> printer{legacy};

    using Handler = std::function<void(const OrderCreatedEvent&)>;
    EventBus<OrderCreatedEvent, Handler> events;
    std::vector<DrinkOrder> kitchen;
    int sales = 0;

    events.subscribe([&kitchen](const OrderCreatedEvent& e) {
```

```
        kitchen.push_back(e.order);
    });
    events.subscribe([&sales](const OrderCreatedEvent&) {
        ++sales;
    });
    events.subscribe([](const OrderCreatedEvent& e) {
        std::cout << "Confirmado: " << e.order.customer_name << "\n";
    });
```

Confirmar y comprobar los efectos

```
    OrderService<decltype(printer), Handler> service{printer, events};
    service.confirm(order);

    std::cout << kitchen.size() << " pedidos en cocina, "
              << sales << " ventas\n"; // 1 y 1.
}
```

`legacy`, `printer` y `events` deben existir mientras el servicio los use. Las variables capturadas por referencia deben existir mientras se ejecuten sus callbacks. Todos los objetos del ejemplo se usan dentro del mismo `main`.

Esqueleto C++20
Completar las declaraciones y los miembros privados necesarios. Las implementaciones template deben estar disponibles en el header. La impresión suministrada es una base que debe extenderse para incluir los campos opcionales.

```
#include <cstddef>
#include <functional>
#include <ostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

enum class Size { Small, Medium, Large };

struct DrinkOrder {
    std::string customer_name;
    std::string drink;
    Size size{};
    std::vector<std::string> extras;
    bool oat_milk = false;
};

struct OrderCreatedEvent {
    DrinkOrder order;
};

void validate(const DrinkOrder& order);

template<class T, class OrderType>
```

```
concept TicketPrinter =
    requires(T printer, const OrderType& order) {
        printer.print(order);
    };

template<class T>
class Builder {
    T value;
public:
    template<class Field, class V>
    Builder& set(Field T::* field, V&& value) &;

    template<class Field, class V>
    Builder&& set(Field T::* field, V&& value) &&;

    T build() &&;
};

class LegacyTicketPrinter {
    std::ostream& output;
public:
    explicit LegacyTicketPrinter(std::ostream& output) : output{output} {}

    void plain_old_format(std::string_view text) const {
        output << "[LEGACY-TICKET]\n" << text << "\n";
    }
};

template<class AdaptedTicketPrinter, class OrderType>
struct TicketPrinterAdapter {
    AdaptedTicketPrinter& legacy;

    void print(const OrderType& order) const {
        auto size_text = [](Size size) -> std::string_view {
            switch (size) {
                case Size::Small: return "small";
                case Size::Medium: return "medium";
                case Size::Large: return "large";
            }
            return "unknown";
        };

        std::ostringstream text;
        text << "TICKET\n"
             << "Client: " << order.customer_name << "\n"
             << "Beverage: " << order.drink << "\n"
             << "Size: " << size_text(order.size);

        legacy.plain_old_format(text.str());
    }
};

template<class Event, class HandlerType>
class EventBus {
public:
```

```
    std::size_t subscribe(HandlerType handler);
    void publish(const Event& event) const;
};

template<TicketPrinter<DrinkOrder> Printer, class HandlerType>
class OrderService {
    Printer& printer;
    EventBus<OrderCreatedEvent, HandlerType>& events;
public:
    OrderService(Printer& printer, EventBus<OrderCreatedEvent, HandlerType>&
events)
        : printer{printer}, events{events} {}

    void confirm(const DrinkOrder& order) {
        validate(order);
        printer.print(order);
        events.publish(OrderCreatedEvent{order});
    }
};
```

Contratos de implementación

Builder

- `set` recibe un puntero a miembro y asigna el valor con reenvío. Las dos sobrecargas permiten encadenar llamadas sobre variables y temporales.
- `build() &&` consume el builder y mueve el producto. Para `DrinkOrder`, invoca `validate` antes de devolverlo.
- Construir un pedido no imprime ni genera eventos; esos efectos pertenecen a `confirm`.

Adapter

- `print(order)` arma el texto y llama una vez a `legacy.plain_old_format(text)`.
- Conservar los campos base `Client:`, `Beverage:` y `Size:` del código suministrado. Añadir `Oat milk: yes/no` y `Extras:`, con los extras en su orden o `none` si no hay.
- El adaptador no publica eventos. Otra impresora que cumpla `TicketPrinter<DrinkOrder>` debe poder sustituirlo sin cambiar el servicio.

Observer y servicio

- `subscribe` guarda el handler y devuelve un identificador único durante la vida del bus. Rechazar handlers vacíos con `std::invalid_argument`.
- `publish` llama una vez a cada suscripción en orden de registro. Publicar sin suscriptores es válido.
- Para delimitar el ejercicio, los callbacks no lanzan, no modifican suscripciones ni generan publicaciones anidadas.
- No se exige cancelación de suscripciones. El identificador retornado sirve para distinguirlas.
- El servicio guarda referencias a impresora y bus; no los crea ni los copia. No debe conocer la clase de la impresora antigua ni los observadores concretos.

Reglas mínimas

- `OrderService` debe ser template sobre cualquier `TicketPrinter<DrinkOrder>`.
- `OrderService` no debe conocer `LegacyTicketPrinter`.
- Los observadores se registran en `EventBus<OrderCreatedEvent, HandlerType>`, no se llaman manualmente desde `main`.
- Construir el pedido con `Builder<DrinkOrder>` y `.set(&DrinkOrder::campo, valor)`.
- `build()` debe consumir el builder con `build() &&`.

Pruebas sugeridas
Incluir `"orders.h"` y `<cassert>`. Las pruebas deben llamar al código implementado y ejecutarse desde el `main` del ejecutable de pruebas.

Builder: pedido completo

```
void test_builder_create_full_order() {
    auto order = Builder<DrinkOrder>{}
        .set(&DrinkOrder::customer_name, "Ana")
        .set(&DrinkOrder::drink, "Latte")
        .set(&DrinkOrder::size, Size::Large)
        .set(&DrinkOrder::oat_milk, true)
        .set(&DrinkOrder::extras, std::vector<std::string>{"cinnamon"})
        .build();

    assert(order.customer_name == "Ana");
    assert(order.drink == "Latte");
    assert(order.oat_milk);
    assert(order.extras.size() == 1);
}
```

Adapter: texto enviado a impresora legacy

```
void test_adapter_send_message_to_legacy() {
    auto order = Builder<DrinkOrder>{}
        .set(&DrinkOrder::customer_name, "Ana")
        .set(&DrinkOrder::drink, "Latte")
        .set(&DrinkOrder::size, Size::Large)
        .build();

    struct FakeLegacyTicketPrinter {
        std::string last_text;
        void plain_old_format(std::string_view text) {
            last_text = text;
        }
    };

    FakeLegacyTicketPrinter legacy;
    TicketPrinterAdapter<FakeLegacyTicketPrinter, DrinkOrder> adapter{legacy};

    adapter.print(order);

    assert(legacy.last_text.find("Client: Ana") != std::string::npos);
    assert(legacy.last_text.find("Beverage: Latte") != std::string::npos);
}
```

Observer: notificación a múltiples suscriptores

```
void test_observer_notify_two_subscribers() {
    auto order = Builder<DrinkOrder>{}
        .set(&DrinkOrder::customer_name, "Ana")
        .set(&DrinkOrder::drink, "Latte")
        .set(&DrinkOrder::size, Size::Large)
        .build();

    using HandlerType = std::function<void(const OrderCreatedEvent&)>;
```

```
    EventBus<OrderCreatedEvent, HandlerType> bus;
    int count = 0;

    bus.subscribe([&](const OrderCreatedEvent&) { ++count; });
    bus.subscribe([&](const OrderCreatedEvent&) { ++count; });
    bus.publish(OrderCreatedEvent{order});

    assert(count == 2);
}
```

Integración: pedido confirmado de extremo a extremo

```
void test_integration_confirm_order() {
    auto order = Builder<DrinkOrder>{}
        .set(&DrinkOrder::customer_name, "Ana")
        .set(&DrinkOrder::drink, "Latte")
        .set(&DrinkOrder::size, Size::Large)
        .set(&DrinkOrder::extras, std::vector<std::string>{"cinnamon"})
        .build();

    struct FakeLegacyTicketPrinter {
        std::string last_text;
        void plain_old_format(std::string_view text) {
            last_text = text;
        }
    };
    FakeLegacyTicketPrinter legacy;

    using HandlerType = std::function<void(const OrderCreatedEvent&)>;
    using PrinterType = TicketPrinterAdapter<FakeLegacyTicketPrinter, DrinkOrder>;
    PrinterType printer{legacy};
    EventBus<OrderCreatedEvent, HandlerType> bus;

    int notifications = 0;
    bus.subscribe([&](const OrderCreatedEvent&) { ++notifications; });
    bus.subscribe([&](const OrderCreatedEvent&) { ++notifications; });

    OrderService<PrinterType, HandlerType> service{printer, bus};
    service.confirm(order);

    assert(legacy.last_text.find("Client: Ana") != std::string::npos);
    assert(legacy.last_text.find("Beverage: Latte") != std::string::npos);
    assert(notifications == 2);
}
```

Casos adicionales

- Pedido sin extras: construir e imprimir con `Extras: none` y leche de avena desactivada.
- Pedido con extras y leche de avena: comprobar todos los campos del ticket.
- Rechazar cliente, bebida o extra vacío, sin impresión ni notificación.
- Confirmar con una impresora falsa que cumpla el concepto y no use el adaptador.
- Verificar orden de los observadores, bus sin suscripciones y rechazo de handler vacío.
- Usar una impresora falsa que lance y comprobar que no se notifique.
- Confirmar dos veces y comprobar dos impresiones y dos eventos.

El archivo de pruebas debe llamar todas las funciones de prueba. Para las cuatro anteriores:

```cpp
int main() {
    test_builder_create_full_order();
    test_adapter_send_message_to_legacy();
    test_observer_notify_two_subscribers();
    test_integration_confirm_order();
}
```

Organización y entrega
Usar `include/orders.h`, `src/main.cpp` y `tests/test_orders.cpp`. Mantener la demostración y las pruebas en ejecutables separados, compilados con C++20. No usar `new`/`delete` directos.

Entregables

- Código C++20 separado en headers claros.
- `main.cpp` con un flujo demostrativo.
- Al menos 3 pruebas con `assert` o framework de pruebas.
- Breve respuesta: ¿qué patrón desacopla qué parte del diseño?

Ejercicio 2: Gastos personales

Escenario de uso
Se desea construir una herramienta de consola para revisar gastos personales del mes. Cada gasto tiene fecha, categoría, descripción y monto. El usuario debe poder ordenar la lista con distintos criterios y exportarla en el formato que necesite.

El diseño debe permitir agregar nuevos formatos de exportación, nuevos criterios de ordenamiento y pasos adicionales sobre la salida, como auditoría o resumen, sin modificar el flujo principal del programa.

La entrega se realizará en GitHub Classroom. El repositorio debe compilar y ejecutar sus pruebas automáticamente con GitHub Actions cada vez que el alumno haga `push` o abra un `pull_request`.

Cómo funciona
El programa recibe una lista de gastos ficticios definidos en `main`. Primero la ordena con una regla elegida y luego la exporta, opcionalmente añadiendo resumen y auditoría.

- **Gasto (`Expense`):** reúne fecha, categoría, descripción y monto.
- **Lista (`ExpenseList`):** conserva los gastos que se van a ordenar y exportar.
- **Ordenador (`sort_with`):** usa el comparador recibido para ordenar la lista.
- **Estrategias:** comparadores por fecha, categoría o monto, implementados como lambdas.
- **Exportadores:** `CsvExporter`, `JsonExporter` y `TextExporter` transforman la lista en una cadena.
- **Decoradores:** `SummaryExporter<Inner>` añade el total y `AuditedExporter<Inner>` añade una marca de auditoría.
- **Fábrica (`make_exporter<E>`):** construye el exportador y retorna un `ExportFunction` que oculta su tipo concreto.

Ordenar, componer y exportar
`sort_with(expenses, by_amount)` modifica el orden de la lista. Luego, `make_exporter<E>()` crea una función que conserva el exportador por valor. Al llamar a esa función con la lista, se ejecuta el exportador o la cadena de decoradores elegida.

Con montos 18.50, 3.20 y 45.00, el orden descendente es 45.00, 18.50, 3.20. La exportación respeta ese orden y el resumen informa un total de 66.70.

Diagrama de referencia y roles

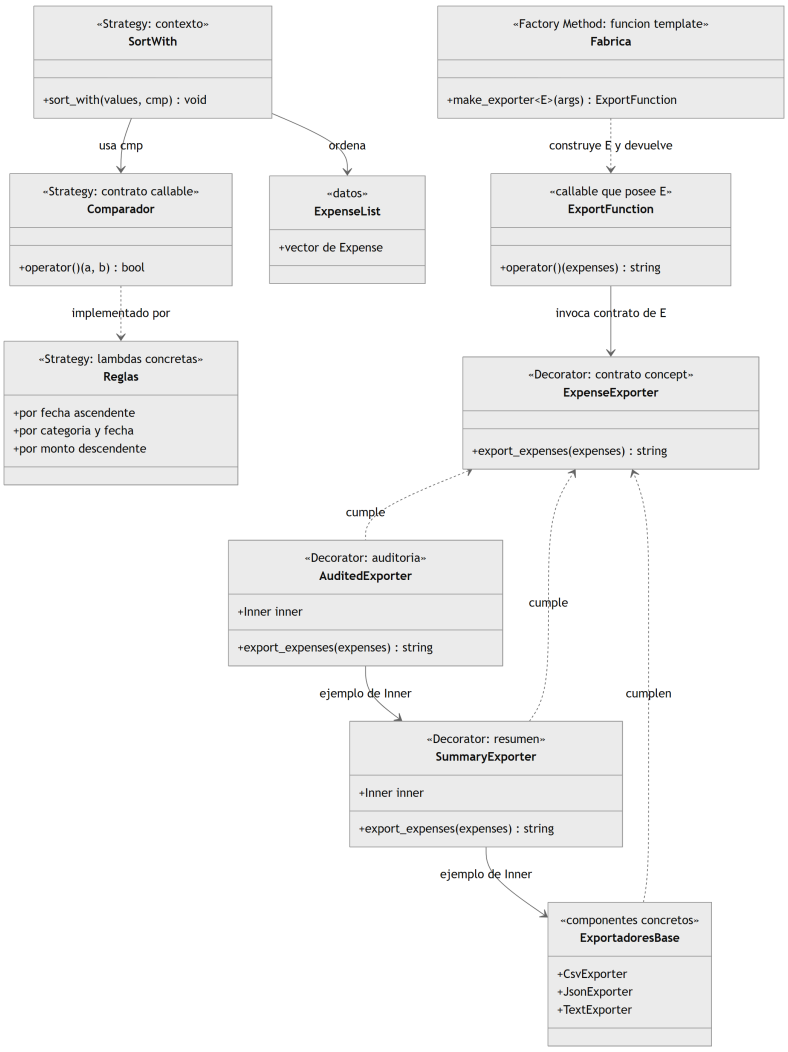


Figura 2: Strategy decide el orden; la fábrica prepara un exportador; Decorator añade contenido a su salida.

- **Strategy:** `sort_with` es el contexto, el concepto `SortStrategy` comprueba la compatibilidad del rango y comparador, y las lambdas son estrategias concretas. Cambiar el comparador cambia el ordenamiento sin reescribir `sort_with`.
- **Factory Method:** `make_exporter<E>` es la fábrica tipada solicitada, `E` es el producto concreto y `ExportFunction` es la interfaz invocable resultante. Se sigue la variante con templates de la sesión; no se requiere la jerarquía virtual de la formulación clásica.
- **Decorator:** `ExpenseExporter` es el contrato común. Los exportadores base son componentes concretos; auditoría y resumen son decoradores que guardan un `Inner` y ofrecen la misma operación.

Los recuadros de conceptos, funciones y lambdas muestran roles de diseño; no exigen clases adicionales ni herencia.

Para qué sirve cada patrón
Strategy separa el algoritmo de ordenamiento de su criterio. La fábrica oculta la construcción y ofrece una llamada uniforme. Decorator permite combinar efectos sobre la salida sin agregar variantes a cada exportador base.

Patrones requeridos

`Factory Method`	Crea exportadores `csv`, `json` o `text`.
`Decorator`	Envuelve un exportador con `AuditedExporter<T>` o `SummaryExporter<T>`.
`Strategy`	Ordena gastos por fecha, categoría o monto.

Reglas del dominio

- Los datos del ejemplo ya son válidos: fechas ISO de ancho fijo `YYYY-MM-DD`, categoría y descripción no vacías, montos finitos y no negativos. No se exige un lector de archivos ni validación de calendario.
- Para delimitar la serialización inicial, usar textos sin comas, comillas, barras inversas ni saltos de línea. El escape de esos caracteres queda como ampliación.
- Orden por fecha: ascendente; por categoría: ascendente, con fecha ascendente como desempate; por monto: descendente.
- Los comparadores deben definir un orden débil estricto: no usar `<=` ni `>=`. No se exige estabilidad entre gastos equivalentes.
- `sort_with` modifica la lista. Los exportadores y decoradores la reciben por referencia constante y no la modifican.
- Serializar montos y totales con dos decimales y punto decimal. Se conserva `double` del esqueleto; no se exige una biblioteca monetaria.
- Una lista vacía se puede ordenar y exportar. CSV conserva su cabecera, JSON devuelve `[]` y texto devuelve una cadena vacía.

Funcionamiento completo antes de implementar

Cargar y elegir una estrategia

```cpp
#include "expenses.h"
#include <iostream>

int main() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20},
        {"2026-06-03", "books", "cpp", 45.00}
    };

    auto by_amount = [](const Expense& a, const Expense& b) {
        return a.amount > b.amount;
    };
    sort_with(expenses, by_amount);
    // Primero books: 45.00.
```

Componer un exportador y pasarlo por la fábrica

Continuación del mismo `main`:

```cpp
    auto decorated = AuditedExporter{
        SummaryExporter{CsvExporter{}}
    };
    auto export_data =
        make_exporter<decltype(decorated)>(decorated);
```

```
    std::cout << export_data(expenses);
    // Cabecera CSV, filas, total: 66.70 y marca audit.
```

`make_exporter` construye una copia del exportador decorado y la guarda dentro del callable. Cada llamada a `export_data` ejecuta auditoría → resumen → CSV; al retornar, cada decorador añade su propia línea a la salida interna.

Cambiar criterio y formato

```
    auto by_date = [](const Expense& a, const Expense& b) {
        return a.date < b.date;
    };
    sort_with(expenses, by_date);

    export_data = make_exporter<JsonExporter>();
    std::cout << export_data(expenses);
    // JSON sin decoradores; primero el gasto del 1 de junio.
}
```

Cambiar de comparador no cambia los exportadores. Reasignar `export_data` sustituye el formato sin cambiar la forma de invocarlo.

Esqueleto C++20
Completar las declaraciones y agregar auxiliares cuando sea necesario. Las implementaciones template deben estar disponibles en el header.

```
#include <algorithm>
#include <cmath>
#include <concepts>
#include <functional>
#include <iomanip>
#include <iterator>
#include <locale>
#include <ranges>
#include <sstream>
#include <string>
#include <tuple>
#include <utility>
#include <vector>

struct Expense {
    std::string date;      // "2026-06-04"
    std::string category;  // "food", "transport", "books"
    std::string detail;
    double amount{};
};

using ExpenseList = std::vector<Expense>;

template<class T>
concept ExpenseExporter =
    requires(T exporter, const ExpenseList& expenses) {
        { exporter.export_expenses(expenses) } -> std::same_as<std::string>;
    };
```

```cpp
using ExportFunction = std::function<std::string(const ExpenseList&)>;

template<class R, class Cmp>
concept SortStrategy =
    std::ranges::random_access_range<R> &&
    std::sortable<std::ranges::iterator_t<R>, Cmp>;

void sort_with(auto& values, auto cmp)
    requires SortStrategy<decltype(values), decltype(cmp)>;

template<ExpenseExporter E, class... Args>
ExportFunction make_exporter(Args&&... args);

struct CsvExporter {
    std::string export_expenses(const ExpenseList& expenses) const;
};

struct JsonExporter {
    std::string export_expenses(const ExpenseList& expenses) const;
};

struct TextExporter {
    std::string export_expenses(const ExpenseList& expenses) const;
};

template<ExpenseExporter Inner>
struct AuditedExporter {
    Inner inner;
    std::string export_expenses(const ExpenseList& expenses) const;
};

template<ExpenseExporter Inner>
struct SummaryExporter {
    Inner inner;
    std::string export_expenses(const ExpenseList& expenses) const;
};
```

Contratos de implementación

Strategy

- `sort_with(values, cmp)` delega en `std::ranges::sort(values, cmp)`.
- Los comparadores reciben dos gastos y devuelven `bool`; no modifican sus argumentos.
- Un comparador de categoría puede usar `std::tie(a.category, a.date)`. Las fechas ISO de ancho fijo se ordenan lexicográficamente.
- No es obligatorio almacenar los comparadores en `std::function`: el parámetro template permite sustituirlos entre llamadas.

Fábrica

- `make_exporter<E>(args...)` construye un `E` con reenvío de argumentos y retorna una lambda que lo conserva por valor.
- La lambda recibe `const ExpenseList&` y retorna el resultado de `E::export_expenses`.
- `ExportFunction` es `std::function`, por lo que el callable almacenado debe ser copiable. Usar exportadores y decoradores copiables en este ejercicio.

- La misma fábrica debe admitir CSV, JSON, texto y exportadores decorados. Si hay selección por nombre, concentrarla en un único punto o registro.

Exportadores base

- CSV: primera línea `date,category,detail,amount`; después, una fila por gasto con campos en ese orden.
- JSON: arreglo de objetos con claves `date`, `category`, `detail` y `amount`. El monto es un número, sin comillas; no añadir coma después del último objeto.
- Texto: una línea por gasto con fecha, categoría, descripción y monto, separados por `|`.
- Mantener el orden recibido y todos los datos. Usar una locale clásica para asegurar el punto decimal.

Decorator

- Cada decorador guarda su `Inner` por valor y llama una vez a su `export_expenses` con la misma lista.
- `SummaryExporter` conserva la salida interna y añade una línea `total: <suma>`, con dos decimales; para lista vacía, el total es 0.00.
- `AuditedExporter` conserva la salida interna y añade una línea `audit: <cantidad> expenses`. No se requiere fecha del sistema.
- Si la salida interna no termina con salto de línea y no está vacía, insertar uno antes de añadir la nueva línea.
- El orden de composición determina el orden de las líneas finales. La cadena del ejemplo añade primero total y luego audit.
- La salida decorada es un reporte textual que contiene una sección CSV o JSON más líneas adicionales; no se exige que el reporte completo siga siendo CSV o JSON válido. Para exportación pura, usar el componente base.

Reglas mínimas

- El cliente no debe tener `if`/`else` repetidos para crear exportadores.
- La fábrica debe seguir el estilo de la diapositiva: `template<ExpenseExporter E, class... Args>`.
- `make_exporter<E>()` debe retornar un `ExportFunction` que llame a `E::export_expenses`.
- Los decoradores deben aceptar cualquier tipo que cumpla `ExpenseExporter`.
- Los decoradores deben poder encadenarse sin cambiar el código cliente.
- Las estrategias deben ser callables compatibles con `sort_with`.
- `sort_with` debe usar `std::ranges::sort`.
- No usar `new`/`delete` directos; usar valores, RAII y objetos estándar.

Ejemplo de composición:

```
auto exporter =
    AuditedExporter{
        SummaryExporter{
            CsvExporter{}
        }
    };

auto output = exporter.export_expenses(expenses);
```

Estructura esperada del repositorio

El repositorio de GitHub Classroom debe incluir como mínimo:

```
.
├── CMakeLists.txt
├── include/
│   └── expenses.h
├── src/
│   └── main.cpp
├── tests/
│   └── test_expenses.cpp
└── .github/
    └── workflows/
        └── tests.yml
```

El archivo `CMakeLists.txt` debe construir un ejecutable demostrativo y un ejecutable de pruebas. Un ejemplo mínimo es:

```
cmake_minimum_required(VERSION 3.22)
project(expenses_patterns LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(expenses_app
    src/main.cpp
)
target_include_directories(expenses_app PRIVATE include)

add_executable(expenses_tests
    tests/test_expenses.cpp
)
target_include_directories(expenses_tests PRIVATE include)

enable_testing()
add_test(NAME expenses_tests COMMAND expenses_tests)
```

Pipeline de GitHub Actions
Crear el archivo `.github/workflows/tests.yml` con un pipeline equivalente a:

```
name: C++20 tests

on:
  push:
    branches: [main, master]
  pull_request:
  workflow_dispatch:

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4
```

```
      - name: Configure
        run: cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug

      - name: Build
        run: cmake --build build --config Debug

      - name: Run tests
        run: ctest --test-dir build --output-on-failure
```

El pipeline debe quedar en verde para considerar la entrega como compilable. Las pruebas pueden usar `assert`, pero deben ejecutarse desde `ctest`.

Pruebas sugeridas
Las siguientes pruebas son una guía. Incluir `"expenses.h"` y `<cassert>`. Pueden adaptarse siempre que evalúen los mismos contratos de diseño.

Strategy: orden por monto

```
void test_strategy_sort_by_amount_descending() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20},
        {"2026-06-03", "books", "cpp", 45.00}
    };

    auto by_amount = [](const Expense& a, const Expense& b) {
        return a.amount > b.amount;
    };

    sort_with(expenses, by_amount);

    assert(expenses.front().amount == 45.00);
    assert(expenses.back().amount == 3.20);
}
```

Strategy: orden por categoría y fecha

```
void test_strategy_sort_by_category_then_date() {
    ExpenseList expenses{
        {"2026-06-03", "transport", "bus", 3.20},
        {"2026-06-01", "books", "cpp", 45.00},
        {"2026-06-02", "books", "algorithms", 38.00}
    };

    auto by_category_then_date = [](const Expense& a, const Expense& b) {
        return std::tie(a.category, a.date) < std::tie(b.category, b.date);
    };

    sort_with(expenses, by_category_then_date);

    assert(expenses[0].category == "books");
    assert(expenses[0].date == "2026-06-01");
    assert(expenses[2].category == "transport");
}
```

Factory Method: creación de exportador CSV

```
void test_factory_create_csv_exporter() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20}
    };

    auto exporter = make_exporter<CsvExporter>();
    auto output = exporter(expenses);

    assert(output.find("date,category,detail,amount") != std::string::npos);
    assert(output.find("food") != std::string::npos);
    assert(output.find("transport") != std::string::npos);
}
```

Factory Method: creación de exportador JSON

```
void test_factory_create_json_exporter() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50}
    };

    auto exporter = make_exporter<JsonExporter>();
    auto output = exporter(expenses);

    assert(output.find("\"category\"") != std::string::npos);
    assert(output.find("\"food\"") != std::string::npos);
    assert(output.find("\"amount\"") != std::string::npos);
}
```

Decorator: auditoría preserva la exportación interna

```
void test_decorator_audit_preserves_inner_export() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20}
    };

    AuditedExporter exporter{CsvExporter{}};
    auto output = exporter.export_expenses(expenses);

    assert(output.find("date,category,detail,amount") != std::string::npos);
    assert(output.find("transport") != std::string::npos);
    assert(output.find("audit") != std::string::npos);
}
```

Decorator: resumen agrega total

```
void test_decorator_summary_adds_total() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20}
    };

    SummaryExporter exporter{TextExporter{}};
```

```
    auto output = exporter.export_expenses(expenses);

    assert(output.find("food") != std::string::npos);
    assert(output.find("total") != std::string::npos);
    assert(output.find("21.70") != std::string::npos);
}
```

Integración: exportación completa de gastos

```
void test_integration_export_sorted_expenses() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20},
        {"2026-06-03", "books", "cpp", 45.00}
    };

    auto by_amount = [](const Expense& a, const Expense& b) {
        return a.amount > b.amount;
    };

    sort_with(expenses, by_amount);

    auto exporter =
        AuditedExporter{
            SummaryExporter{
                CsvExporter{}
            }
        };

    auto output = exporter.export_expenses(expenses);

    assert(expenses.front().amount == 45.00);
    assert(output.find("date,category,detail,amount") != std::string::npos);
    assert(output.find("total") != std::string::npos);
    assert(output.find("audit") != std::string::npos);
}
```

El archivo `tests/test_expenses.cpp` debe llamar todas las pruebas desde `main`:

```
int main() {
    test_strategy_sort_by_amount_descending();
    test_strategy_sort_by_category_then_date();
    test_factory_create_csv_exporter();
    test_factory_create_json_exporter();
    test_decorator_audit_preserves_inner_export();
    test_decorator_summary_adds_total();
    test_integration_export_sorted_expenses();
}
```

Casos adicionales

- Lista vacía y lista de un elemento: ordenar y verificar la salida de los tres formatos.
- CSV con cabecera exacta y JSON con arreglo, claves y montos numéricos, sin coma final.
- Orden por fecha y empate de categoría resuelto por fecha; no exigir orden entre elementos equivalentes.

- Decoradores anidados en ambos órdenes: mismos gastos y total, distinto orden de las líneas de cierre.
- Comprobar que exportar no cambie los datos ni su orden.
- Crear exportadores de texto y decorados mediante `make_exporter`, además de CSV y JSON.
- Verificar exactamente dos decimales y punto decimal en montos y total.

Entregables

- Código C++20 separado en headers claros.
- `main.cpp` con un flujo demostrativo de carga, ordenamiento y exportación.
- `tests/test_expenses.cpp` con al menos 5 pruebas ejecutadas por `ctest`.
- `.github/workflows/tests.yml` con el pipeline de GitHub Actions.
- Captura o enlace al resultado exitoso del workflow en GitHub Classroom.
- Breve respuesta en `README.md`: ¿qué patrón desacopla qué parte del diseño?

Evaluación

Criterio	Puntaje
Patrones pedidos aplicados correctamente	6 pts
Uso idiomático de C++20: concepts, templates, ranges, RAII	4 pts
Diseño desacoplado y componentes pequeños	4 pts
Pruebas automatizadas ejecutadas por GitHub Actions	4 pts
Explicación breve de decisiones de diseño	2 pts

Ejercicio 3: Sensores inteligentes

Escenario de uso
Se desea construir un sistema simple para monitorear sensores de aulas. Cada sensor produce una lectura de temperatura, CO2 o ruido. Para este ejercicio, las lecturas se simulan dentro del programa, sin hardware real ni comunicación externa.

El diseño debe permitir agregar nuevos sensores, adaptar proveedores externos, cambiar reglas de alerta y notificar a distintos componentes cuando una lectura sea riesgosa.

La entrega se realizará en GitHub Classroom. El repositorio debe compilar y ejecutar sus pruebas automáticamente con GitHub Actions cada vez que el alumno haga `push` o abra un `pull_request`.

Cómo funciona
El programa simula una ronda de medición de un aula. Para comenzar, se usa un sensor de CO2 del aula `L-301`, con advertencia desde 900 ppm y nivel crítico desde 1200 ppm.

- **Configuración (`SensorConfig`):** indica tipo, aula, proveedor, umbrales y frecuencia. Se construye con `Builder<SensorConfig>`.
- **Sensor:** produce una `Reading` con tipo, valor y aula. El concepto `Sensor` exige una operación `read()`.
- **Adaptador:** convierte la respuesta numérica de `VendorProbe::sample()` en una `Reading`.
- **Decoradores:** validan y registran la lectura sin cambiar el contrato `read()`.
- **Fábrica (`make_sensor<S>`):** construye el sensor concreto y devuelve un `SensorFn` para invocarlo sin conocer su tipo.
- **Monitor:** coordina una lectura, consulta la estrategia y publica la alerta cuando corresponde. Puede ser una función; no se exige una clase.
- **Estrategia (`AlertStrategy`):** clasifica la lectura como normal, advertencia o crítica.
- **Bus (`EventBus`):** comunica `AlertRaised` a panel, auditoría y mensajes mediante callbacks.

Registrar observadores y procesar lecturas
Primero, `main` registra los callbacks con `bus.subscribe(handler)`. Cada handler recibe `const AlertRaised&` y retorna `void`. Todos reciben cada alerta publicada; este ejercicio no exige filtros ni cancelación de suscripciones.

Luego, el monitor obtiene una lectura con `sensor()`, llama a `strategy(reading, config)` y, si el nivel es distinto de `Normal`, ejecuta `bus.publish({reading, level})`. Los observadores se ejecutan desde el bus.

Lectura de CO2	Regla normal	Resultado
850 ppm	Normal	Se lee y registra; no se publica alerta.
950 ppm	Warning	Se publica una alerta a todos los observadores.
1300 ppm	Critical	Se publica una alerta crítica a todos los observadores.

Cambiar la estrategia a estricta hace que 850 ppm produzca `Warning`, porque el umbral de advertencia pasa a 810 ppm. Esto no requiere cambiar el sensor, el bus ni las suscripciones.

Diagramas de referencia y roles
Los diagramas separan la preparación de sensores de la clasificación y notificación para mantener legibles los seis patrones. Los recuadros de funciones, conceptos y callbacks son referencias de diseño; no implican clases adicionales ni herencia virtual.

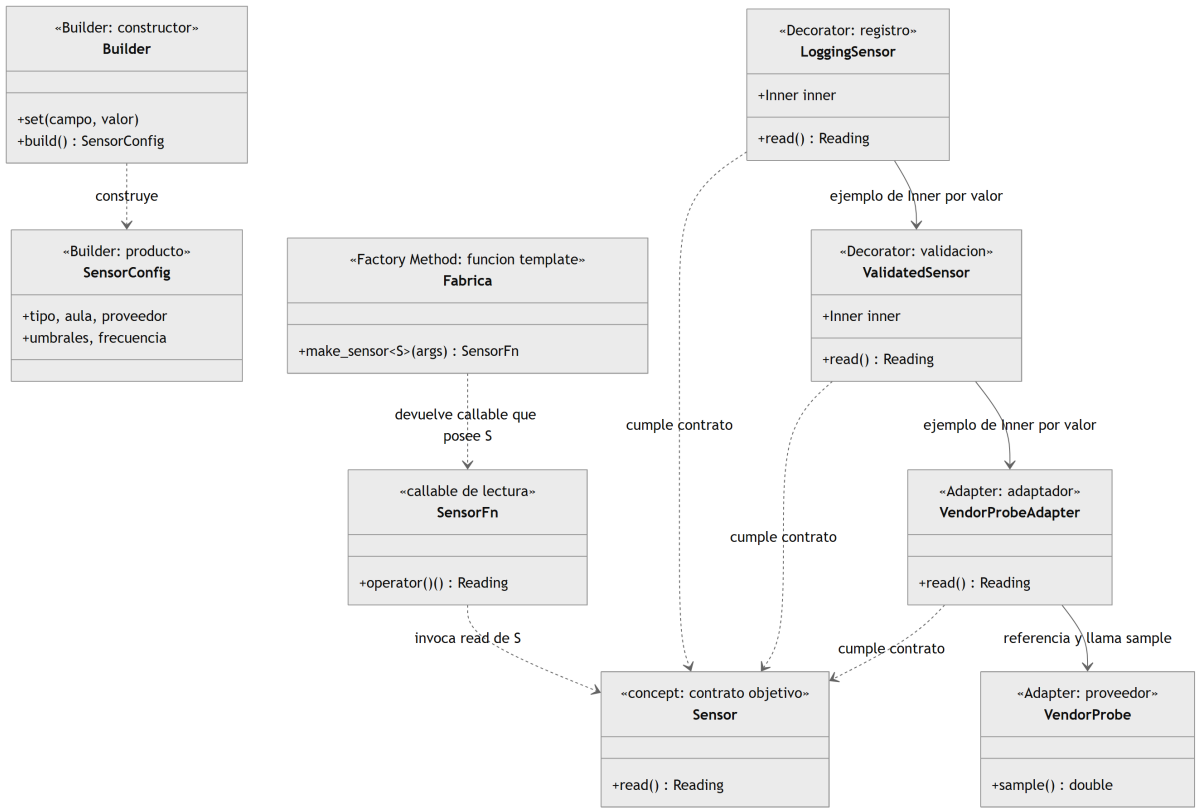


Figura 3: Creación y lectura: Builder prepara la configuración; la fábrica construye sensores; Adapter y Decorator conservan el contrato Sensor.

- **Builder:** `Builder<SensorConfig>` es el constructor y `SensorConfig` es el producto; `main` encadena los pasos.
- **Factory Method:** `make_sensor<S>` es la fábrica tipada solicitada; `S` es el sensor concreto y `SensorFn` es la interfaz invocable resultante. Se conserva el estilo de la sesión basado en templates, sin exigir la jerarquía virtual de la variante clásica.
- **Adapter:** `Sensor` es el contrato objetivo, `VendorProbe` es el proveedor incompatible y `VendorProbeAdapter` es el adaptador.
- **Decorator:** `LoggingSensor<Inner>` y `ValidatedSensor<Inner>` envuelven otro sensor por valor y ofrecen el mismo `read()`. Pueden encadenarse.

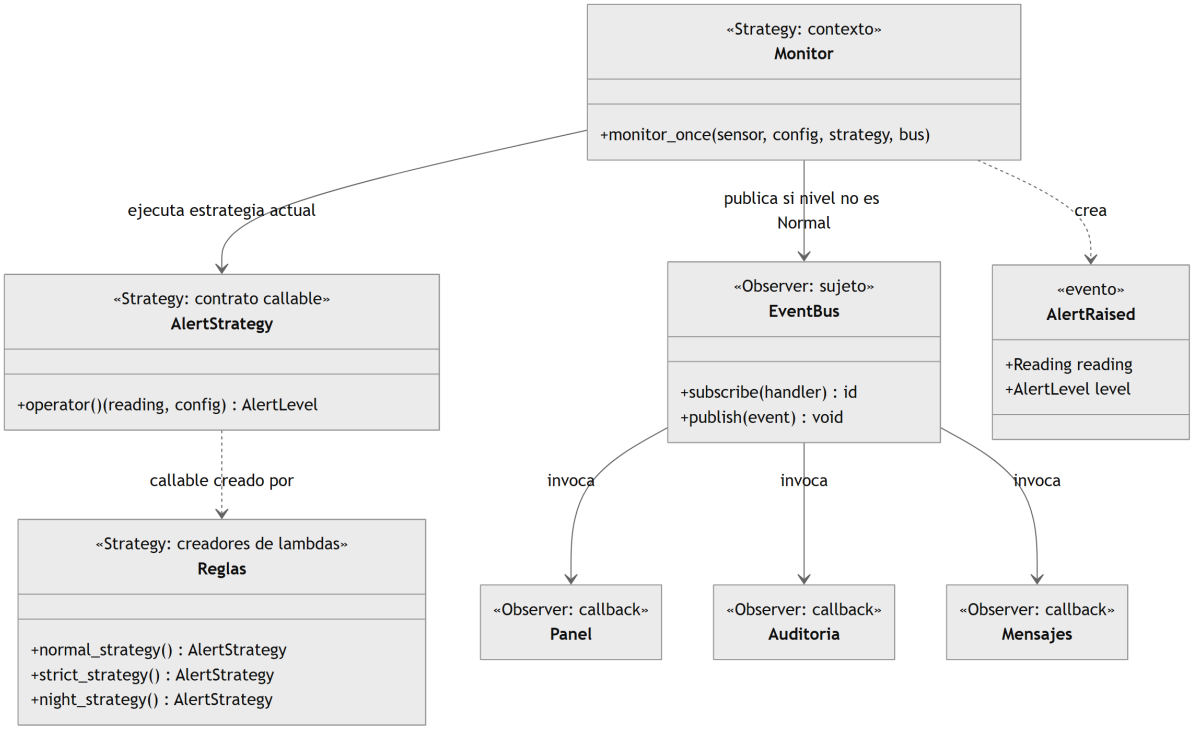


Figura 4: Comportamiento: el monitor usa una Strategy y publica eventos mediante Observer.

- **Strategy:** el monitor es el contexto; `AlertStrategy` es el contrato común; las lambdas normal, estricta y nocturna son las estrategias concretas. Reasignar la variable `strategy` sustituye la regla usada en la siguiente ronda.
- **Observer:** `EventBus` es el sujeto que conserva suscripciones; panel, auditoría y mensajes son los observadores. `AlertRaised` es el dato comunicado. El monitor publica sin llamar directamente a los observadores.

Para qué sirve cada patrón

Builder reúne la configuración; la fábrica oculta cómo se construye cada sensor; Adapter incorpora un proveedor con otra interfaz; Decorator añade validación y registro. Strategy permite sustituir la clasificación y Observer permite agregar receptores de alertas sin modificar el monitor.

Patrones requeridos

`Factory Method`	Crea sensores `temperature`, `co2` o `noise`.
`Builder`	Construye `SensorConfig`: tipo, aula, proveedor, umbrales y frecuencia.
`Adapter`	Adapta `VendorProbe` al contrato `Sensor`.
`Decorator`	Agrega `LoggingSensor<T>` o `ValidatedSensor<T>`.
`Strategy`	Clasifica lecturas con regla normal, estricta o nocturna.
`Observer`	Publica `AlertRaised` hacia panel, auditoría y mensajes.

Reglas del dominio

- Una lectura contiene tipo de sensor, valor y aula. Usar grados Celsius para temperatura, ppm para CO2 y dB para ruido.
- El aula y el proveedor de la configuración deben ser no vacíos. Los umbrales deben ser finitos y cumplir `warning_threshold < critical_threshold`; la frecuencia debe ser positiva.
- Toda lectura debe tener un aula no vacía y un valor finito. Temperatura no puede ser menor que −273.15 °C; CO2 y ruido no pueden ser negativos.

- La lectura procesada debe corresponder al tipo y aula de su configuración. El monitor rechaza discrepancias con `std::invalid_argument` antes de clasificar o publicar.
- En cada ronda se realiza una sola lectura y una sola clasificación. Una lectura inválida no genera alertas.
- Un resultado `Normal` no se publica. `Warning` o `Critical` genera exactamente un `AlertRaised` por ronda, aunque coincida con el de la ronda anterior.
- La frecuencia es un dato de configuración: no se requieren temporizadores ni esperas reales.

Funcionamiento completo antes de implementar
El siguiente `main` muestra cómo se usarán las declaraciones del esqueleto una vez completadas. `monitor_once` y las tres funciones que crean estrategias se declaran más adelante como parte del trabajo.

Configurar y componer un sensor

```
#include "sensors.h"
#include <iostream>
#include <vector>

int main() {
    auto config = Builder<SensorConfig>{}
        .set(&SensorConfig::type, SensorType::Co2)
        .set(&SensorConfig::room, "L-301")
        .set(&SensorConfig::provider, "vendor-x")
        .set(&SensorConfig::warning_threshold, 900.0)
        .set(&SensorConfig::critical_threshold, 1200.0)
        .set(&SensorConfig::sample_rate_seconds, 5)
        .build();

    VendorProbe vendor{850.0};
    auto decorated = LoggingSensor{
        ValidatedSensor{
            VendorProbeAdapter{vendor, config.type, config.room}
        }
    };
    auto sensor = make_sensor<decltype(decorated)>(decorated);
```

La fábrica guarda su sensor por valor dentro del callable. El adaptador conserva una referencia a `vendor`: ese proveedor debe vivir mientras el sensor pueda utilizarse. `sensor()` ejecuta la cadena: logging → validación → adaptador → proveedor.

Conectar observadores
Continuación del mismo `main`:

```
    using Handler = std::function<void(const AlertRaised&)>;
    EventBus<AlertRaised, Handler> bus;
    std::vector<AlertRaised> audit;
    int messages = 0;

    bus.subscribe([](const AlertRaised& e) {
        std::cout << e.reading.room << ": "
                  << e.reading.value << "\n";
    });
    bus.subscribe([&audit](const AlertRaised& e) {
        audit.push_back(e);
```

```
    });
    bus.subscribe([&messages](const AlertRaised&) {
        ++messages;
    });
```

Las suscripciones se conservan en el bus. `audit` y `messages` deben seguir existiendo mientras se invoquen los callbacks que los capturan.

Leer y sustituir la estrategia

```
    AlertStrategy strategy = normal_strategy();
    monitor_once(sensor, config, strategy, bus);
    // 850 ppm: Normal. No se publica.

    strategy = strict_strategy();
    monitor_once(sensor, config, strategy, bus);
    // Misma lectura: Warning. Se ejecutan los 3 callbacks.

    strategy = night_strategy();
    monitor_once(sensor, config, strategy, bus);
    // Para CO2, la regla nocturna conserva los umbrales normales.

    std::cout << audit.size() << " alertas, "
              << messages << " mensajes\n"; // 1 y 1.
}
```

`monitor_once` recibe por referencia la estrategia actual y el bus. Cada llamada usa la función guardada en `strategy` en ese momento; el bus no decide si una lectura es peligrosa.

Esqueleto C++20
Completar las declaraciones y agregar los miembros privados necesarios. Las implementaciones template deben estar disponibles en el header.

```
#include <cmath>
#include <concepts>
#include <cstddef>
#include <functional>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

enum class SensorType { Temperature, Co2, Noise };
enum class AlertLevel { Normal, Warning, Critical };

struct Reading {
    SensorType type{};
    double value{};
    std::string room;
};

struct SensorConfig {
    SensorType type{};
    std::string room;
```

```
    std::string provider;
    double warning_threshold{};
    double critical_threshold{};
    int sample_rate_seconds{};
};

struct AlertRaised {
    Reading reading;
    AlertLevel level{};
};

template<class T>
class Builder {
    T value;
public:
    template<class Field, class V>
    Builder& set(Field T::* field, V&& value) &;

    template<class Field, class V>
    Builder&& set(Field T::* field, V&& value) &&;

    T build() &&;
};

template<class T>
concept Sensor =
    requires(T sensor) {
        { sensor.read() } -> std::same_as<Reading>;
    };

using SensorFn = std::function<Reading()>;

template<Sensor S, class... Args>
SensorFn make_sensor(Args&&... args);

using AlertStrategy =
    std::function<AlertLevel(const Reading&, const SensorConfig&)>;

class VendorProbe {
    double value;
public:
    explicit VendorProbe(double value) : value{value} {}
    double sample() const;
};

struct VendorProbeAdapter {
    VendorProbe& probe;
    SensorType type{};
    std::string room;

    Reading read() const;
};

template<Sensor Inner>
struct LoggingSensor {
```

```
    Inner inner;
    Reading read();
};

template<Sensor Inner>
struct ValidatedSensor {
    Inner inner;
    Reading read();
};

template<class Event, class HandlerType>
class EventBus {
public:
    std::size_t subscribe(HandlerType handler);
    void publish(const Event& event) const;
};
```

Declaraciones para coordinar el monitor

Agregar al mismo header, después de `EventBus`:

```
using AlertHandler = std::function<void(const AlertRaised&)>;
using AlertBus = EventBus<AlertRaised, AlertHandler>;

void validate(const SensorConfig& config);
void validate(const Reading& reading);

AlertStrategy normal_strategy();
AlertStrategy strict_strategy();
AlertStrategy night_strategy();

void monitor_once(
    SensorFn& sensor,
    const SensorConfig& config,
    const AlertStrategy& strategy,
    const AlertBus& bus
);
```

Contratos de Builder y fábrica

- `set` asigna el campo apuntado por el puntero a miembro y permite encadenar llamadas sobre lvalues y temporales.
- `build() &&` mueve el resultado; para `SensorConfig`, valida antes de devolverlo y lanza `std::invalid_argument` si la configuración es inválida.
- `make_sensor<S>(args...)` construye un `S` con los argumentos reenviados, lo conserva por valor y retorna una lambda que llama a su `read()`. Usar una lambda mutable para admitir lecturas no constantes.
- Como `SensorFn` es `std::function`, el callable almacenado debe ser copiable. Para este ejercicio, usar sensores y decoradores copiables.
- Proporcionar sensores simulados para temperatura, CO2 y ruido. Pueden recibir una `Reading` fija al construirse. La fábrica debe permitir crearlos y también construir el sensor adaptado o decorado.

Contratos de Adapter y Decorator

- `VendorProbe::sample()` retorna el valor simulado recibido por su constructor.

- El adaptador llama una vez a `sample()` y añade tipo y aula para formar la lectura, sin clasificar ni publicar.
- `ValidatedSensor::read()` obtiene una lectura, llama a `validate(reading)` y la devuelve sin modificarla. Ante lectura inválida, `validate(reading)` lanza `std::logic_error`.
- `LoggingSensor::read()` obtiene una lectura, escribe tipo, aula y valor en consola y devuelve la misma lectura. Si el sensor interno lanza, propaga la excepción.
- En `LoggingSensor{ValidatedSensor{adapted}}`, una lectura inválida se rechaza antes de registrarse. Invertir el orden cambia ese comportamiento.
- Los decoradores añaden comportamiento a `read()`; no evalúan estrategias ni publican eventos.

Contratos de Strategy
- Todas las estrategias reciben `const Reading&` y `const SensorConfig&`, y retornan `AlertLevel`.
- La regla normal compara primero el umbral crítico y luego el de advertencia; incluye la igualdad en ambos límites.
- La regla estricta multiplica ambos umbrales por 0.9, para todos los tipos de sensor.
- La regla nocturna multiplica ambos umbrales por 0.8 solo para ruido; temperatura y CO2 mantienen los umbrales normales.
- Las funciones `normal_strategy()`, `strict_strategy()` y `night_strategy()` crean y devuelven esas lambdas. No modifican la configuración original.
- El contexto es `monitor_once`; debe invocar la estrategia recibida sin codificar dentro de él las tres reglas.

Contratos de Observer y del monitor
- `subscribe` guarda el callback y devuelve un identificador único durante la vida del bus. Rechazar handlers vacíos con `std::invalid_argument`.
- `publish` ejecuta una vez cada suscripción en orden de registro. Publicar sin suscriptores es válido.
- Para delimitar el ejercicio, los callbacks no lanzan, no registran suscripciones ni generan publicaciones anidadas mientras se notifica.
- `monitor_once` valida la configuración y comprueba que sensor y estrategia no estén vacíos; en caso contrario lanza `std::invalid_argument` antes de leer.
- Después, obtiene una sola lectura, la valida, comprueba que coincida con la configuración, consulta la estrategia y publica únicamente niveles distintos de `Normal`.
- Las referencias guardadas por adaptadores y callbacks deben seguir siendo válidas mientras se usen. No se exige cancelación de suscripciones en este ejercicio.

Reglas mínimas
- El monitor no debe saber si el sensor es interno, simulado o adaptado desde un proveedor externo.
- La fábrica debe seguir el estilo de la diapositiva: `template<Sensor S, class... Args>`.
- `make_sensor<S>()` debe retornar un `SensorFn` que llame a `S::read`.
- `Builder<SensorConfig>` debe construir la configuración usando `.set(&SensorConfig::campo, valor)`.
- `build()` debe consumir el builder con `build() &&`.
- Puede usarse un mapa `std::unordered_map<std::string, SensorFn>` para elegir sensores por configuración.
- Los decoradores deben aceptar cualquier tipo que cumpla `Sensor` y deben poder encadenarse.
- La estrategia de alerta debe ser intercambiable desde `main`.
- Los observadores se registran en `EventBus<AlertRaised, HandlerType>`, no se llaman manualmente desde el monitor.
- No usar `new`/`delete` directos; usar valores, RAII y objetos estándar.

Ejemplo de composición:

```
VendorProbe vendor{1300};
VendorProbeAdapter adapted{vendor, SensorType::Co2, "L-301"};
LoggingSensor sensor{
    ValidatedSensor{
        adapted
    }
};

auto reading = sensor.read();
```

Estructura esperada del repositorio
El repositorio de GitHub Classroom debe incluir como mínimo:

```
.
├── CMakeLists.txt
├── include/
│   └── sensors.h
├── src/
│   └── main.cpp
├── tests/
│   └── test_sensors.cpp
└── .github/
    └── workflows/
        └── tests.yml
```

El archivo `CMakeLists.txt` debe construir un ejecutable demostrativo y un ejecutable de pruebas. Un ejemplo mínimo es:

```
cmake_minimum_required(VERSION 3.22)
project(sensors_patterns LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(sensors_app
    src/main.cpp
)
target_include_directories(sensors_app PRIVATE include)

add_executable(sensors_tests
    tests/test_sensors.cpp
)
target_include_directories(sensors_tests PRIVATE include)

enable_testing()
add_test(NAME sensors_tests COMMAND sensors_tests)
```

Pipeline de GitHub Actions
Crear el archivo `.github/workflows/tests.yml` con un pipeline equivalente a:

```
name: C++20 tests
```

```yaml
on:
  push:
    branches: [main, master]
  pull_request:
  workflow_dispatch:

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Configure
        run: cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug

      - name: Build
        run: cmake --build build --config Debug

      - name: Run tests
        run: ctest --test-dir build --output-on-failure
```

El pipeline debe quedar en verde para considerar la entrega como compilable. Las pruebas pueden usar `assert`, pero deben ejecutarse desde `ctest`.

Pruebas sugeridas
Las siguientes pruebas son una guía. Pueden adaptarse siempre que evalúen los mismos contratos de diseño.

Builder: configuración válida

```
void test_builder_create_valid_config() {
    auto config = Builder<SensorConfig>{}
        .set(&SensorConfig::type, SensorType::Co2)
        .set(&SensorConfig::room, "L-301")
        .set(&SensorConfig::provider, "vendor-x")
        .set(&SensorConfig::warning_threshold, 900.0)
        .set(&SensorConfig::critical_threshold, 1200.0)
        .set(&SensorConfig::sample_rate_seconds, 5)
        .build();

    assert(config.type == SensorType::Co2);
    assert(config.room == "L-301");
    assert(config.provider == "vendor-x");
    assert(config.critical_threshold == 1200.0);
}
```

Factory Method: creación de sensor simulado

```
void test_factory_create_sensor_function() {
    struct FakeSensor {
        Reading reading;
        Reading read() const {
            return reading;
```

```
        }
    };

    auto sensor = make_sensor<FakeSensor>(
        Reading{SensorType::Temperature, 24.5, "L-101"}
    );

    auto reading = sensor();

    assert(reading.type == SensorType::Temperature);
    assert(reading.value == 24.5);
    assert(reading.room == "L-101");
}
```

Adapter: proveedor externo convertido a Sensor

```
void test_adapter_convert_vendor_probe_to_sensor() {
    VendorProbe vendor{1300.0};
    VendorProbeAdapter adapted{vendor, SensorType::Co2, "L-301"};

    auto reading = adapted.read();

    assert(reading.type == SensorType::Co2);
    assert(reading.value == 1300.0);
    assert(reading.room == "L-301");
}
```

Decorator: lectura preservada

```
void test_decorator_preserve_reading() {
    struct FakeSensor {
        Reading reading;
        Reading read() const {
            return reading;
        }
    };

    FakeSensor fake{Reading{SensorType::Noise, 70.0, "L-202"}};
    LoggingSensor decorated{fake};

    auto reading = decorated.read();

    assert(reading.type == SensorType::Noise);
    assert(reading.value == 70.0);
    assert(reading.room == "L-202");
}
```

Decorator: validación rechaza lectura inválida

```
void test_decorator_validate_invalid_reading() {
    struct FakeSensor {
        Reading reading;
        Reading read() const {
            return reading;
        }
```

```
    };

    FakeSensor fake{Reading{SensorType::Temperature, -300.0, "L-404"}};
    ValidatedSensor decorated{fake};

    bool failed = false;
    try {
        decorated.read();
    } catch (const std::logic_error&) {
        failed = true;
    }

    assert(failed);
}
```

Strategy: clasificación crítica

```
void test_strategy_detect_critical_alert() {
    auto config = Builder<SensorConfig>{}
        .set(&SensorConfig::type, SensorType::Co2)
        .set(&SensorConfig::room, "L-301")
        .set(&SensorConfig::provider, "vendor-x")
        .set(&SensorConfig::warning_threshold, 900.0)
        .set(&SensorConfig::critical_threshold, 1200.0)
        .set(&SensorConfig::sample_rate_seconds, 5)
        .build();

    Reading reading{SensorType::Co2, 1300.0, "L-301"};

    AlertStrategy normal = normal_strategy();

    assert(normal(reading, config) == AlertLevel::Critical);
}
```

Strategy: regla estricta cambia el umbral

```
void test_strategy_strict_rule_detects_warning_earlier() {
    SensorConfig config{
        SensorType::Co2,
        "L-301",
        "vendor-x",
        900.0,
        1200.0,
        5
    };

    Reading reading{SensorType::Co2, 850.0, "L-301"};

    AlertStrategy strict = strict_strategy();

    assert(strict(reading, config) == AlertLevel::Warning);
}
```

Observer: alerta recibida

```
void test_observer_receive_alert() {
    using HandlerType = std::function<void(const AlertRaised&)>;
    EventBus<AlertRaised, HandlerType> bus;
    bool notified = false;

    bus.subscribe([&](const AlertRaised& event) {
        notified = event.level == AlertLevel::Critical &&
                   event.reading.room == "L-301";
    });

    bus.publish({
        Reading{SensorType::Co2, 1300.0, "L-301"},
        AlertLevel::Critical
    });

    assert(notified);
}
```

Integración: lectura crítica notificada

```
void test_integration_notify_critical_sensor_alert() {
    auto config = Builder<SensorConfig>{}
        .set(&SensorConfig::type, SensorType::Co2)
        .set(&SensorConfig::room, "L-301")
        .set(&SensorConfig::provider, "vendor-x")
        .set(&SensorConfig::warning_threshold, 900.0)
        .set(&SensorConfig::critical_threshold, 1200.0)
        .set(&SensorConfig::sample_rate_seconds, 5)
        .build();

    VendorProbe vendor{1300.0};
    VendorProbeAdapter adapted{vendor, SensorType::Co2, "L-301"};
    LoggingSensor sensor{
        ValidatedSensor{
            adapted
        }
    };

    AlertStrategy normal = normal_strategy();

    using HandlerType = std::function<void(const AlertRaised&)>;
    EventBus<AlertRaised, HandlerType> bus;
    bool notified = false;

    bus.subscribe([&](const AlertRaised& event) {
        notified = event.level == AlertLevel::Critical;
    });

    auto read_sensor = make_sensor<decltype(sensor)>(sensor);
    monitor_once(read_sensor, config, normal, bus);

    assert(notified);
}
```

El archivo `tests/test_sensors.cpp` debe llamar todas las pruebas desde `main`:

```
int main() {
    test_builder_create_valid_config();
    test_factory_create_sensor_function();
    test_adapter_convert_vendor_probe_to_sensor();
    test_decorator_preserve_reading();
    test_decorator_validate_invalid_reading();
    test_strategy_detect_critical_alert();
    test_strategy_strict_rule_detects_warning_earlier();
    test_observer_receive_alert();
    test_integration_notify_critical_sensor_alert();
}
```

Casos adicionales e integración del monitor
La prueba anterior usa el monitor real. Ampliar la cobertura con los siguientes casos:

- Con 850 ppm y regla normal, no se notifica; al cambiar a estricta, la misma lectura produce una sola advertencia por suscriptor.
- Probar igualdad con los umbrales, valores inmediatamente inferiores y un resultado crítico.
- Con ruido y umbrales 60/80, una lectura de 65 es `Warning` con la regla normal y `Critical` con la nocturna; CO2 conserva su clasificación nocturna.
- Rechazar configuración inválida, NaN, infinito, temperatura menor que −273.15, CO2 o ruido negativos y aula vacía.
- Rechazar lectura de otra aula o tipo antes de publicar. Verificar que una lectura inválida no llegue a los observadores.
- Comprobar dos observadores, orden de ejecución, publicación sin observadores y rechazo de handlers vacíos.
- Crear los tres sensores simulados mediante la fábrica y probar dos decoradores encadenados.
- Verificar que el sensor se lee una vez por ronda y que repetir una lectura riesgosa publica otra alerta.

Todas las pruebas agregadas deben llamarse desde el `main` de pruebas. Incluir `"sensors.h"` y `<cassert>` en ese archivo.

Entregables
- Código C++20 separado en headers claros.
- `main.cpp` con un flujo demostrativo de configuración, lectura, clasificación y notificación.
- `tests/test_sensors.cpp` con al menos 6 pruebas ejecutadas por `ctest`.
- `.github/workflows/tests.yml` con el pipeline de GitHub Actions.
- Captura o enlace al resultado exitoso del workflow en GitHub Classroom.
- Breve respuesta en `README.md`: ¿qué patrón desacopla qué parte del diseño?

Evaluación

Criterio	Puntaje
Patrones pedidos aplicados correctamente	6 pts
Uso idiomático de C++20: concepts, templates, ranges, RAII	4 pts
Diseño desacoplado y componentes pequeños	4 pts
Pruebas automatizadas ejecutadas por GitHub Actions	4 pts
Explicación breve de decisiones de diseño	2 pts

Evaluación

Criterio	Puntaje
Patrones pedidos aplicados correctamente	6 pts
Uso idiomático de C++20: concepts, templates, ranges, RAII	4 pts
Diseño desacoplado y componentes pequeños	4 pts
Pruebas y flujo demostrativo	4 pts
Explicación breve de decisiones de diseño	2 pts